Final Project: Extending Ultralytics with a Custom `DoubleConv` Block

Objectives

- To understand how Ultralytics builds models from YAML files
- To add a custom `DoubleConv` block into the Ultralytics codebase
- To expose a custom backbone so it can be constructed from a cfg YAML file
- To train a YOLO model using the custom cfg file
- To collect metrics and inspect training results from Ultralytics outputs

Overview

In this final project, you will extend the local Ultralytics repository at `/home/ralampay/workspace/ultralytics` with a custom `DoubleConv`-style block, then integrate it into a YOLO model through a custom backbone and cfg YAML file.

Ultralytics models are built from YAML files through `parse_model()` in `ultralytics/nn/tasks.py`. If you want a new block or backbone to be usable inside a YAML file, you must do three things:

1. implement the module in the Ultralytics codebase
2. export the module so `tasks.py` can see it
3. teach the YAML parser how to construct it and track its output channels

How the Integration Works

1. Implement the custom block

Create a custom `DoubleConv` block in:

- `/home/ralampay/workspace/ultralytics/ultralytics/nn/modules/block.py`

Example:

```
import torch
import torch.nn as nn

class CustomDoubleConv(nn.Module):
    def __init__(self, c1: int, c2: int):
        super().__init__()
        self.block = nn.Sequential(
            nn.Conv2d(c1, c2, kernel_size=3, padding=1, bias=False),
            nn.BatchNorm2d(c2),
            nn.ReLU(inplace=True),
            nn.Conv2d(c2, c2, kernel_size=3, padding=1, bias=False),
            nn.BatchNorm2d(c2),
```

```
            nn.ReLU(inplace=True),
        )

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return self.block(x)
```

This is the same basic pattern used in many segmentation backbones: two convolution layers with normalization and activation.

2. Wrap the block in a backbone

If you want the block to drive a detection model, you usually need more than one block. You need a backbone that returns multi-scale features such as `P3`, `P4`, and `P5`.

Example backbone:

```
class DoubleConvBackbone(nn.Module):
    def __init__(self, c1: int, c2: int = 1024, out_channels=(256, 512, 1024)):
        super().__init__()
        self.stem = nn.Sequential(
            nn.Conv2d(c1, 64, kernel_size=3, stride=2, padding=1, bias=False),
            nn.BatchNorm2d(64),
            nn.ReLU(inplace=True),
        )

        self.stage1 = CustomDoubleConv(64, 128)
        self.down1 = nn.MaxPool2d(2)
        self.stage2 = CustomDoubleConv(128, 256)
        self.down2 = nn.MaxPool2d(2)
        self.stage3 = CustomDoubleConv(256, 512)
        self.down3 = nn.MaxPool2d(2)
        self.stage4 = CustomDoubleConv(512, 1024)

        self.p3_proj = nn.Conv2d(256, out_channels[0], kernel_size=1)
        self.p4_proj = nn.Conv2d(512, out_channels[1], kernel_size=1)
        self.p5_proj = nn.Conv2d(1024, out_channels[2], kernel_size=1)
        self.c2 = c2

    def forward(self, x: torch.Tensor):
        x = self.stem(x)
        x = self.stage1(x)
        x = self.down1(x)
        p3 = self.stage2(x)
        x = self.down2(p3)
        p4 = self.stage3(x)
        x = self.down3(p4)
```

```
        p5 = self.stage4(x)
        return [self.p3_proj(p3), self.p4_proj(p4), self.p5_proj(p5)]
```

The important point is that the backbone returns a list of feature maps, not classification logits.

3. Export the module

Expose the new names in:

- `/home/ralampay/workspace/ultralytics/ultralytics/nn/modules/__init__.py`

For example, add:

```
from .block import CustomDoubleConv, DoubleConvBackbone
```

and include them in `__all__`.

If the module is not exported here, `parse_model()` cannot resolve it from the YAML.

4. Register parser behavior in `tasks.py`

Ultralytics YAML rows use this format:

```
[from, repeats, module, args]
```

If your backbone constructor needs the input channel count inserted automatically, add a branch in:

- `/home/ralampay/workspace/ultralytics/ultralytics/nn/tasks.py`

Example:

```
elif m is DoubleConvBackbone:
    c1, c2 = ch[f], args[0]
    args = [c1, *args]
```

This tells the parser to prepend the current input channel count `c1` before constructing the backbone. It also records `c2` as the parser-facing output width for that layer.

5. Use `Index` in the YAML

If the backbone returns multiple feature maps, the YAML should select them with `Index`.

Conceptually:

```
image
  -> DoubleConvBackbone
  -> [P3, P4, P5]
  -> Index
  -> YOLO neck/head
```

Tasks

Part 1: Create the Custom `DoubleConv` Block

Create a block named `CustomDoubleConv`.

Minimum requirements:

- it must inherit from `nn.Module`
- it must accept input and output channel arguments
- it must preserve spatial size when stride is 1
- it must support a forward pass on tensors shaped `(N, C, H, W)`

You may place it in:

- `ultralytics/nn/modules/block.py`

Part 2: Create a Custom Backbone

Create a backbone that uses your `CustomDoubleConv` block repeatedly and returns `P3`, `P4`, and `P5`.

Minimum requirements:

- it must produce three feature maps for detection
- it must return a Python list
- the output channels should match the expected YOLO head inputs

Expected output pattern:

```
def forward(self, x):
    ...
    return [p3, p4, p5]
```

Part 3: Export the New Classes

Update:

- `ultralytics/nn/modules/__init__.py`

Make sure both of these are exported:

- `CustomDoubleConv`
- `DoubleConvBackbone`

Part 4: Register the Backbone in `tasks.py`

Update:

- `ultralytics/nn/tasks.py`

Add:

1. an import for `DoubleConvBackbone`

2. a parser branch for `DoubleConvBackbone`

Pattern:

```
elif m is DoubleConvBackbone:
    c1, c2 = ch[f], args[0]
    args = [c1, *args]
```

Part 5: Create a cfg YAML File with a Custom Backbone

Create a model cfg file such as:

- `ultralytics/cfg/models/ext/doubleconv-yolo.yaml`

Example:

```
nc: 80
end2end: True
reg_max: 1

scales:
  n: [0.50, 0.25, 1024]
  s: [0.50, 0.50, 1024]
  m: [0.50, 1.00, 512]
  l: [1.00, 1.00, 512]
  x: [1.00, 1.50, 512]

backbone:
  - [-1, 1, DoubleConvBackbone, [1024, [256, 512, 1024]]]
  - [0, 1, Index, [256, 0]]
  - [0, 1, Index, [512, 1]]
  - [0, 1, Index, [1024, 2]]
  - [-1, 1, SPPF, [1024, 5, 3, True]]
  - [-1, 2, C2PSA, [1024]]

head:
  - [-1, 1, nn.Upsample, [None, 2, "nearest"]]
  - [[-1, 2], 1, Concat, [1]]
  - [-1, 2, C3k2, [512, True]]

  - [-1, 1, nn.Upsample, [None, 2, "nearest"]]
  - [[-1, 1], 1, Concat, [1]]
  - [-1, 2, C3k2, [256, True]]

  - [-1, 1, Conv, [256, 3, 2]]
  - [[-1, 8], 1, Concat, [1]]
  - [-1, 2, C3k2, [512, True]]
```

```
  - [-1, 1, Conv, [512, 3, 2]]
  - [[-1, 5], 1, Concat, [1]]
  - [-1, 1, C3k2, [1024, True, 0.5, True]]

  - [[11, 14, 17], 1, Detect, [nc]]
```

Your YAML should clearly show:

- the custom backbone name
- the arguments passed to it
- the indexed P3, P4, and P5 outputs

Part 6: Train the Model Using the cfg File

From the Ultralytics repository root:

```
cd /home/ralampay/workspace/ultralytics
pip install -e .
```

Sample Python training code:

```
from ultralytics import YOLO

model = YOLO("ultralytics/cfg/models/ext/doubleconv-yolo.yaml")

results = model.train(
    data="ultralytics/cfg/datasets/coco8.yaml",
    epochs=10,
    imgsz=640,
    batch=16,
    device=0,
    project="runs/final_project",
    name="doubleconv_exp",
    plots=True,
)
```

Sample CLI training command:

```
cd /home/ralampay/workspace/ultralytics
yolo detect train \
  model=ultralytics/cfg/models/ext/doubleconv-yolo.yaml \
  data=ultralytics/cfg/datasets/coco8.yaml \
  epochs=10 \
  imgsz=640 \
  batch=16 \
  project=runs/final_project \
  name=doubleconv_cli \
  plots=True
```

You may replace the dataset YAML with your own dataset path.

Part 7: Take Metrics and See the Results

Ultralytics automatically saves metrics and run artifacts.

After training, inspect:

- `runs/final_project/doubleconv_exp/results.csv`
- `runs/final_project/doubleconv_exp/weights/best.pt`
- `runs/final_project/doubleconv_exp/weights/last.pt`
- `runs/final_project/doubleconv_exp/results.png`
- `runs/final_project/doubleconv_exp/confusion_matrix.png`
- `runs/final_project/doubleconv_exp/confusion_matrix_normalized.png`

A. Validate the best checkpoint

```
from ultralytics import YOLO

model = YOLO("runs/final_project/doubleconv_exp/weights/best.pt")
metrics = model.val(data="ultralytics/cfg/datasets/coco8.yaml")

print(metrics.results_dict)
print("mAP50:", metrics.box.map50)
print("mAP50-95:", metrics.box.map)
print("precision:", metrics.box.mp)
print("recall:", metrics.box.mr)
```

B. Print the metrics returned by training

```
from ultralytics import YOLO

model = YOLO("ultralytics/cfg/models/ext/doubleconv-yolo.yaml")
results = model.train(
    data="ultralytics/cfg/datasets/coco8.yaml",
    epochs=10,
    imgsz=640,
    batch=16,
    project="runs/final_project",
    name="doubleconv_exp_2",
)

print(results.results_dict)
print(results.box.map50)
print(results.box.map)
```

C. Read the epoch-by-epoch CSV

```
from pathlib import Path
import pandas as pd
```

```
csv_path = Path("runs/final_project/doubleconv_exp/results.csv")
df = pd.read_csv(csv_path)

print(df.columns.tolist())
print(df.tail())
```

Typical fields to report:

- training loss values
- validation loss values
- `metrics/precision(B)`
- `metrics/recall(B)`
- `metrics/mAP50(B)`
- `metrics/mAP50-95(B)`

D. Inspect per-class metrics

```
from ultralytics import YOLO

model = YOLO("runs/final_project/doubleconv_exp/weights/best.pt")
metrics = model.val(data="ultralytics/cfg/datasets/coco8.yaml")

for row in metrics.summary():
    print(row)
```

Part 8: Verify that the Custom Backbone Is Really Being Used

Do not assume the YAML is correct just because training starts.

At minimum:

1. print the model summary
2. confirm that `DoubleConvBackbone` appears
3. run a dummy forward pass

Example:

```
from ultralytics import YOLO
import torch

model = YOLO("ultralytics/cfg/models/ext/doubleconv-yolo.yaml")
model.model.info()

x = torch.randn(1, 3, 640, 640)
y = model.model(x)
print(type(y))
```

Common integration errors:

- the module was not exported in `nn/modules/__init__.py`
- `tasks.py` does not import the module
- `parse_model()` does not handle the constructor arguments correctly
- the backbone returns the wrong number of feature maps
- the feature-map channels do not match the YOLO head

Required Output

Submit the following:

1. your custom `DoubleConv` block code
2. your custom backbone code
3. your custom model cfg YAML file
4. sample training code
5. the parser change in `tasks.py`
6. training and validation metrics
7. a short discussion of the saved plots and results

Recommended Report Structure

Your report should include:

- a short description of your custom `DoubleConv` block
- the files you modified
- the exact YAML you used
- the dataset used for training
- the training arguments used
- the final `mAP50` and `mAP50-95`
- the location of `best.pt`
- screenshots or discussion of `results.png` and confusion matrix plots

Notes

Important implementation notes:

- `parse_model()` in `ultralytics/nn/tasks.py` is the core place that turns YAML rows into real modules
- a custom backbone that returns multiple features should usually be paired with `Index` layers in YAML
- the detection head only works cleanly if the backbone output channels and strides match what the head expects
- Ultralytics training automatically saves `results.csv`, `best.pt`, and validation plots inside the run directory

The point of this lab is not just to make training run. The point is to understand how Ultralytics turns a custom Python block into a YAML-defined detection model.